



Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

NFC/RFID COM + APP

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 23 de abril de 2026

Índice

Introducción	3
Descripción de la tecnología RFID/NFC	4
Usos actuales de la tecnología	4
Funcionamiento y descripción del Módulo Sensor utilizado (RFID-RC522)	5
Circuitos	6
1. Receptor	6
a) Diseño	6
b) Código Arduino IDE	8
c) Código Aplicación en Python	11
Conclusiones	15
Enlace a GitHub y Video	15



Introducción

El presente sistema implementa una solución de automatización para la gestión de libros en una biblioteca mediante el uso de tecnología RFID y comunicación serial con un microcontrolador ESP32-S3 Mini. A través de un lector RFID, se capturan identificadores únicos (UID) asociados a cada libro, los cuales son enviados al sistema desarrollado en Python. Este sistema integra una interfaz gráfica construida con CustomTkinter y una base de datos SQLite, permitiendo consultar en tiempo real la información de los libros registrados. La arquitectura combina procesamiento embebido y software de escritorio, logrando una interacción eficiente entre hardware y software para la validación y visualización de datos.



Descripción de la tecnología RFID/NFC

La tecnología RFID (Radio Frequency Identification) identifica objetos, personas o tarjetas mediante ondas de radio. Su función principal es almacenar y transmitir datos de forma inalámbrica entre un tag y un lector. Esta tecnología no necesita contacto físico ni una línea de visión directa, por lo que resulta práctica y rápida para control de acceso, inventarios, rastreo de objetos y sistemas de identificación.

Por otro lado, NFC (Near Field Communication) es una evolución o variante de RFID de alta frecuencia. Su principal característica es su funciona a muy corta distancia, normalmente de unos pocos centímetros. NFC permite una comunicación más directa entre dos dispositivos, por ejemplo entre un teléfono celular y una tarjeta, o entre dos teléfonos. A diferencia de RFID, NFC puede trabajar en modos más interactivos, como lectura, emulación de tarjeta y comunicación entre dispositivos.

Usos actuales de la tecnología

La tecnología RFID/NFC se utiliza tanto en la vida cotidiana como en la industria debido a su rapidez, comodidad y automatización.

Casos comunes de RFID:

- Control de inventarios en almacenes y comercios.
- Seguimiento de productos en logística y transporte.
- Identificación de animales mediante microchips.
- Control de acceso en edificios, empresas o escuelas.
- Sistemas de peaje y estacionamientos automatizados.
- Gestión de libros en bibliotecas.

Casos comunes de NFC:

- Pagos sin contacto con tarjetas o teléfonos móviles.
- Acceso a transporte público.
- Intercambio rápido de datos entre dispositivos.
- Validación de identidad o autenticación.
- Lectura de etiquetas inteligentes en publicidad, museos o productos.



Funcionamiento y descripción del Módulo Sensor utilizado (RFID-RC522)

El RFID-RC522 es un módulo lector de tarjetas de radiofrecuencia inalámbrico muy utilizado en proyectos de electrónica y sistemas de identificación. Trabaja con una frecuencia de 13.56 MHz y permite leer la información contenida en tarjetas o llaveros RFID compatibles. Detecta una etiqueta cercana, lee su código de identificación único y envía esa información al microcontrolador, como un ESP32 o Arduino, para que realice una acción específica.

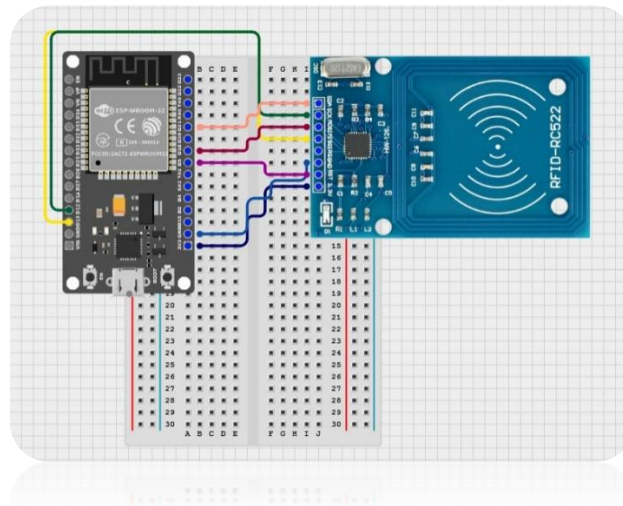
El RFID-RC522 cuenta con varios pines de conexión, entre los que destacan SDA/SS, SCK, MOSI, MISO, RST, VCC y GND. Estos pines permiten la comunicación con la placa controladora y la alimentación del módulo. Generalmente, se alimenta con 3.3V, ya que no es recomendable conectarlo directamente a 5V.

En un proyecto, este módulo suele emplearse para sistemas de acceso, control de asistencia, identificación de usuarios, automatización de puertas y lectura de credenciales. Gracias a su tamaño reducido, bajo consumo y facilidad de integración, el RFID-RC522 es una opción práctica para aplicaciones de identificación sin contacto.

Circuitos

1. Receptor

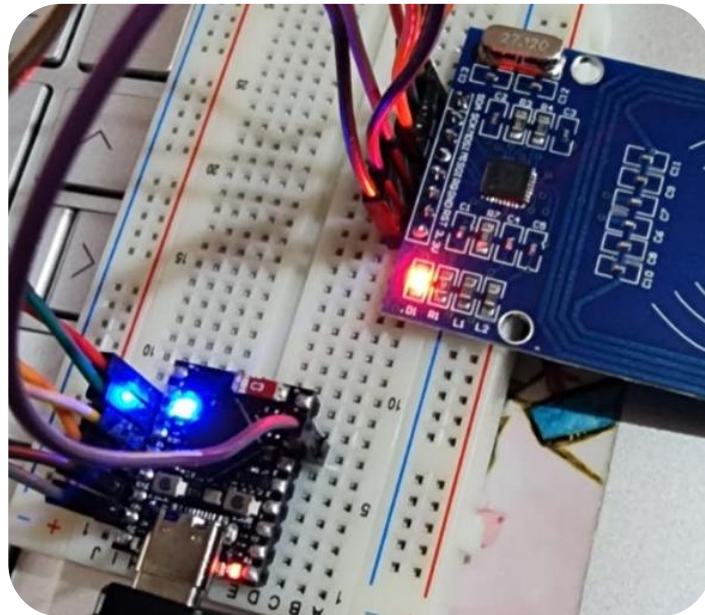
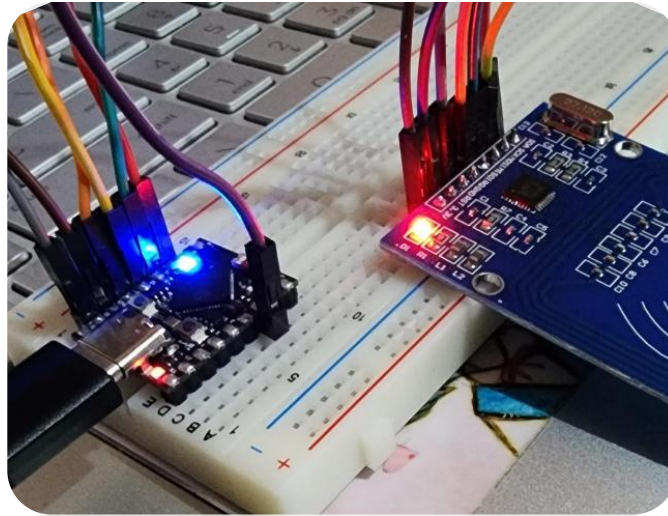
a) Diseño



El diagrama muestra otro tipo de esp32, por lo que la imagen cambia los pines 10 y 13 por 18 y 19

Conexión de pines 1 a 1	
RC522	ESP32-S3 Super Mini
VCC	3.3V
RST	GPIO 5
GND	GND
IRQ	No conectado
MISO	GPIO 13
MOSI	GPIO 11
SCK	GPIO 12
SDA (SS)	GPIO 10





Este circuito permanece idéntico al avance anterior, donde el diagrama sigue siendo una adaptación del circuito real con la ESP32 s3. Los cables, componentes y su orden no han cambiado así como su código en ArduinoIDE, las diferencias ahora radican en el código de Python y la creación de una base de datos que, por ahora, es compacta y se ha preparado para extrapolarla a un ambiente más exigente y profesional una vez se provea más información que conformen una base de datos más realista de una biblioteca.

b) Código Arduino IDE

Librerías

```
#include <MFRC522v2.h>
#include <MFRC522DriverSPI.h>
#include <MFRC522DriverPinSimple.h>
```

Estas librerías permiten usar el lector MFRC522 (RFID) con comunicación SPI:

- MFRC522v2.h → lógica principal del lector RFID
- MFRC522DriverSPI.h → comunicación SPI
- MFRC522DriverPinSimple.h → configuración sencilla del pin SS (chip select)

Configuración del lector

```
MFRC522DriverPinSimple ss_pin(10);
MFRC522DriverSPI driver{ss_pin};
MFRC522 mfrc522{driver};
```

Aquí estás creando los objetos necesarios:

- Define el pin **SS (Slave Select)** en el pin 10, que activará el lector RFID
- Conecta el lector usando SPI
- Y el mfrc522 es el objeto principal para interactuar con el lector

Inicializa SPI

```
SPI.begin(12, 13, 11, 10);
```

Parámetros:

```
SPI.begin(SCK, MISO, MOSI, SS)
```

- 12 → SCK (reloj)
- 13 → MISO (datos hacia ESP32)
- 11 → MOSI (datos hacia lector)
- 10 → SS

Esto depende del cableado del ESP32-S3 mini

Inicializa el lector RFID

```
mfr522.PCD_Init();
```

- Enciende y prepara el lector
- Sin esto, no detecta tarjetas

Detecta tarjeta

```
if (mfr522.PICC_IsNewCardPresent() && mfr522.PICC_ReadCardSerial()) {
```

1. `PICC_IsNewCardPresent()` → detecta si hay una tarjeta nueva
2. `PICC_ReadCardSerial()` → lee su UID

Solo entra si ambas condiciones se cumplen

Construcción del UID

```
String uid = "";
```

- Variable donde se guardará el identificador único

Formato hexadecimal

```
if (mfr522.uid.uidByte[i] < 0x10) uid += "0";
```

```
uid += String(mfr522.uid.uidByte[i], HEX);
```

- Convierte cada byte a **hexadecimal**
- Si el valor es menor a 0x10, agrega un 0 adelante

Envío a la PC

```
Serial.println(uid);
```

Este es el puente con la app de Python:

- Envía SOLO el UID
- Termina con salto de línea (\n)

Esto es justo lo que el código en Python leerá con:

```
ser.readline()
```

Detener comunicación con la tarjeta

```
mfr522.PICC_HaltA();
```

- Le dice al tag que ya terminó su lectura
- Evita lecturas repetidas inmediatas

Pequeña pausa

```
delay(1000);
```

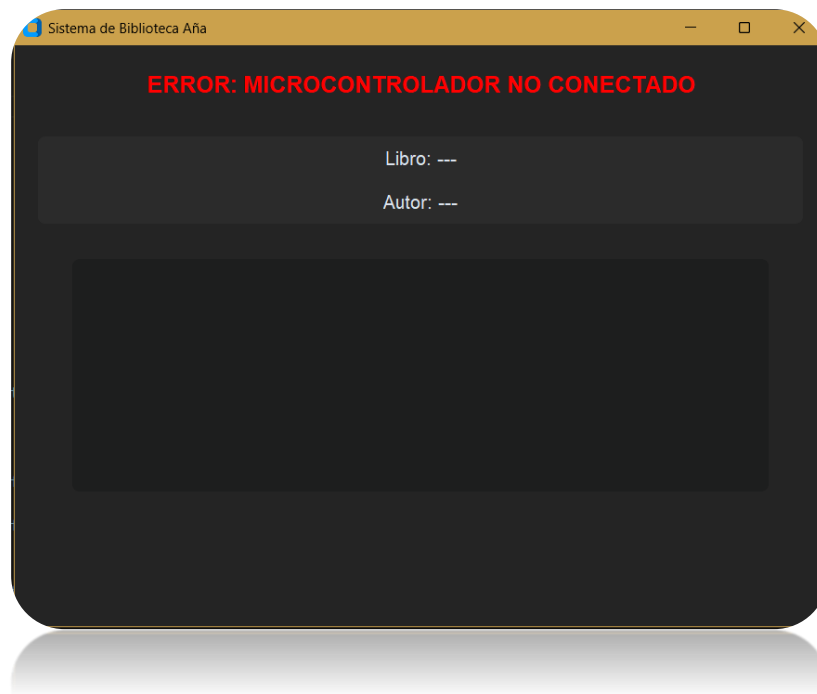
- Espera 1 segundo
- Evita que el mismo libro se registre muchas veces seguidas

El sistema funciona entonces así:

ESP32

1. Detecta tarjeta RFID
2. Lee UID
3. Lo convierte a texto
4. Lo manda por serial

c) Código Aplicación en Python



Debido a la escala de este avance, se optó por recrear la base de datos en el mismo código en Python que ha de ser adaptado, como siempre, acorde a los componentes del usuario.

Para este caso, se escanearon diversas tarjetas bancarias, incluyendo los dos tags con los que viene el RC550 y se ingresaron en la base de datos, para que al ser escaneados por el circuito, devuelvan la información vinculada para cada caso.

Configuración e imports

```
import customtkinter as ctk
import serial
import sqlite3
import threading
```

- customtkinter: interfaz moderna (ventana, labels, textbox).
- serial: comunicación con el ESP32.
- sqlite3: base de datos local.
- threading: permite leer el puerto serial sin congelar la interfaz.

```
PUERTO_COM = 'COM3'
```

- Define el puerto donde está conectado tu ESP32.

```
self.init_db()
```

- inicializa la base de datos cada vez que se abre la app.

```
threading.Thread(target=self.leer_serial, daemon=True).start()
```

- Inicia la lectura del ESP32 en segundo plano.
- daemon=True hace que el hilo muera cuando cierres la app.
- Evita que la GUI se congele.

Base de datos

```
conn = sqlite3.connect('biblioteca.db')
```

- Crea o abre el archivo de base de datos.

```
cursor.execute("DROP TABLE IF EXISTS libros")
```

- Borra la tabla cada vez que inicia el programa.

```
cursor.execute("""CREATE TABLE libros  
    (uid TEXT PRIMARY KEY, titulo TEXT, autor TEXT)""")
```

- Define la estructura:
 - uid = identificador RFID
 - titulo, autor = info del libro

```
cursor.executemany("INSERT INTO libros VALUES (?, ?, ?)", libros_biblioteca)
```

- Inserta todos los libros en una sola operación.

Lectura del ESP32 (clave del sistema)

```
ser = serial.Serial(PUERTO_COM, 115200, timeout=1)
```

- Abre comunicación serial.
- El valor de 115200 debe coincidir con el código en el ESP32 que se esté por utilizar.

```
linea = ser.readline().decode('utf-8').strip()
```

- Lee el UID enviado por el lector RFID.
- `.decode()` convierte bytes → texto.

`self.after(0, self.consultar_libro, linea)`

- Muy importante:
 - Evita modificar la GUI desde otro hilo.
 - Ejecuta consultar libro en el hilo principal (seguro).

`except:`

`self.after(0, lambda: self.label_status.configure(...))`

- Manejo de error si el ESP32 no está conectado.

Consulta del libro

`cursor.execute("SELECT titulo, autor FROM libros WHERE uid = ?", (uid,))`

- Busca el UID en la base de datos.
- Uso de '?' → evita inyección SQL.

`resultado = cursor.fetchone()`

- Obtiene un solo resultado (porque uid es único).

d) Resultados:

`if resultado:`

`self.label_status.configure(text="ACCESO CONCEDIDO", text_color="green")`

- Cambia estado visual.

`self.historial_txt.insert("0.0", f"[OK] Tag: {uid} -> {titulo}\n")`

- Guarda historial, último resultado hasta arriba.

`else:`

`self.label_status.configure(text="ACCESO DENEGADO", text_color="red")`

```
self.historial_txt.insert("0.0", f"[ERROR] IDENTIFICADOR: {uid} no encontrado...")
```

Registra intentos fallidos.

2. Flujo de la aplicación

1. Inicia la app → se crea DB
2. Se abre hilo → escucha ESP32
3. ESP32 manda UID
4. Python recibe UID
5. Busca en SQLite
6. Muestra resultado en GUI



Conclusiones

Ramírez Rodríguez Martín de Jesús:

El desarrollo de este sistema demuestra la viabilidad de integrar tecnologías de identificación por radiofrecuencia con aplicaciones de software para optimizar procesos en entornos bibliotecarios. La comunicación entre el ESP32-S3 Mini y la aplicación en Python permite una identificación rápida y precisa de los libros, mientras que el uso de una base de datos local facilita la gestión de la información. Aunque el prototipo cumple con los objetivos planteados, su estructura puede ampliarse para incluir funcionalidades adicionales como persistencia avanzada de datos, control de usuarios o integración con sistemas en red, lo que lo convierte en una base sólida para desarrollos futuros más complejos.

Santacruz Rodarte Cristian Alexis:

Se pudo realizar con éxito la implementación de RFID para el uso en sistemas de almacenamiento en bibliotecas, por medio de señales y el correcto uso de una base de datos se puede crear un entorno organizado para las bibliotecas que quieran implementar el producto.

Pedroza Pérez Emmanuel:

En esta ocasión se desarrolló un código más completo que use los módulos RFID y código Python. Al tener un UID único se escanea y detecta que tag se esta usando. Se puede ver que se usan conocimientos adquiridos a lo largo de la carrera como tecnologías inalámbricas, programación y bases de datos para luego aplicarlos a una solución de biblioteca para identificar libros, ver su estatus de manera mas sencilla y obtener información respecto al libro escaneado.

Enlace a GitHub y Video

Este enlace que se muestra a continuación, muestra tanto el repositorio donde se encuentra la documentación misma, los códigos empleados para esta práctica y su video de evidencia:

<https://github.com/MartinITAAgs/NFC-RFID-COM-APP>